

Конспект

▼ Метрики якості

▼ Метрики якості в задачах регресії

Метрики якості використовуються для:

1. Тренування моделей. Такі метрики називаються **функціоналом похибки**.
2. Підбору гіперпараметрів моделі. Метрика може бути такою самою, як і при тренуванні, або відрізнятись від неї
3. Оцінки придатності моделі

Найчастіше використовується **середньоквадратична похибка** (MSE). Її досить легко оптимізувати, наприклад, градієнтним спуском

$$MSE(a, X) = \frac{1}{l} \sum_{i=1}^l (a(x_i) - y_i)^2$$

Цей функціонал сильно штрафує за великі помилки. На викидах даних штрафи будуть підводитись в квадрат і, таким чином, модель може підлаштовуватись під викиди даних. Тобто алгоритм може навчатись на тих даних, які не є характерними для датасету

Менш чутливим в даному сенсі є **середня абсолютна похибка** (MAE)

$$MAE(a, X) = \frac{1}{l} \sum_{i=1}^l |a(x_i) - y_i|$$

Однак цей функціонал гірше оптимізувати, оскільки в точці 0 він не має похідної

Коефіцієнт детермінації R^2 показує яка доля дисперсії відповідей пояснюється моделлю. Змінюється від -1 до 1. Прийнятними для моделі є лише позитивні значення. Гарним показником є значення в межах від 0.27 до 0.4 і вище

$$R^2 = 1 - \frac{\sum_{i=1}^l (a(x_i) - y_i)^2}{\sum_{i=1}^l (y_i - \bar{y})^2}$$

▼ Метрики якості в задачах класифікації

Оцінка якості класифікатора полягає у тому, щоб виміряти наскільки точно класифікатор розставив мітки класів. У випадку бінарної класифікації датасет буде містити дві мітки 1 і 0 (або 1 і -1). Можливі всього 4 варіанти комбінації прогнозованого і заданого значень міток класів

		Predicted condition	
		Predicted Positive (PP)	Predicted Negative (PN)
Actual condition	Positive (P)	True positive (TP), hit	False negative (FN), miss, underestimation
	Negative (N)	False positive (FP), false alarm, overestimation	True negative (TN), correct rejection

True positive - класифікатор вірно визначив позитивну мітку класу.

True negative - класифікатор вірно визначив негативну мітку класу.

False positive ще називають **помилкою I роду**. Класифікатор помилково передбачив позитивний клас, хоча насправді це зразок

негативного класу.

False negative називають **помилкою II роду**. Класифікатор не визначив зразок позитивного класу

В залежності від задачі важливіше може бути відсутність помилок I або II роду. Наприклад, в медицині важливо не пропустити пацієнта із хворобою, тому помилок II роду не повинно бути. В кредитному скорінгу помилка I роду це випадок, коли ми хибно визнали некредитоспроможною людину і, відповідно, не отримали прибуток.

Метрика **Accuracy**

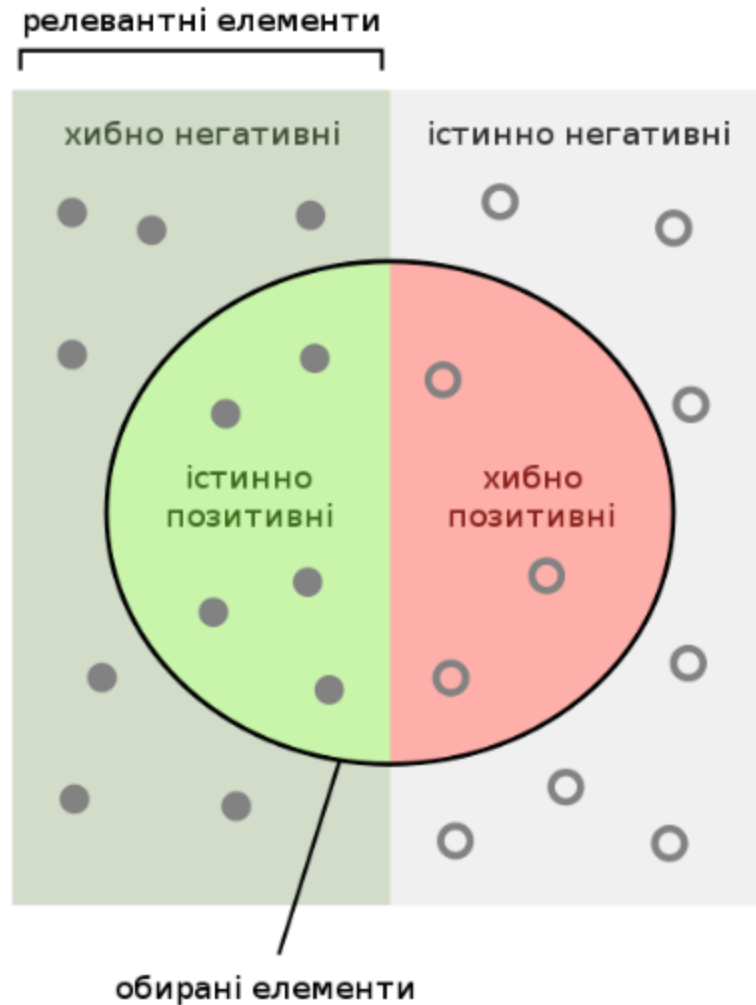
$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN}$$

Метрика Accuracy показує долю правильних відповідей класифікатора. Ця метрика непогано оцінює модель, але вона не розрізняє як точно ми знайшли зразки позитивного класу.

Метрика **Precision**

$$\text{Precision} = \frac{TP}{TP + FP}$$

Метрика Precision (влучність) показує яка частина із того, що класифікатор передбачив як позитивний клас насправді є позитивним класом. У випадку, якщо класифікатор "скупий" і дає невелику кількість точних позитивних прогнозів ця метрика буде давати 1.



Як багато з обраних елементів є релевантними?

Влучність = $\frac{\text{істинно позитивні}}{\text{істинно позитивні} + \text{хибно позитивні}}$

Як багато з релевантних елементів стають обраними?

Повнота = $\frac{\text{істинно позитивні}}{\text{істинно позитивні} + \text{хибно негативні}}$

Але чи всі екземпляри позитивного класу класифікатор знайшов?
Відповідь на це запитання дає метрика **Recall** (повнота):

$$\text{Recall} = \frac{TP}{TP + FN}$$

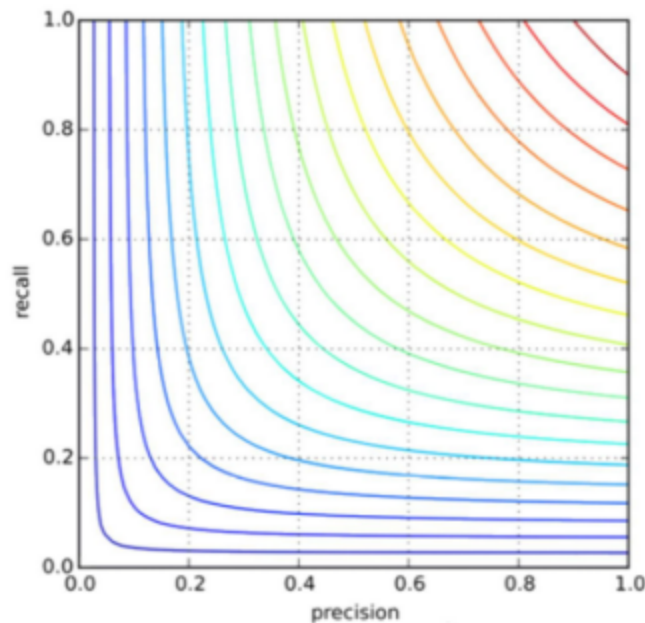
Метрика Recall показує нам яку долю позитивних екземплярів класифікатор передбачив саме як позитивні.

Для різних задач може бути більш важливою або влучність або повнота.

Більш універсальною є метрика **F-score**.

$$F_1 = 2 \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

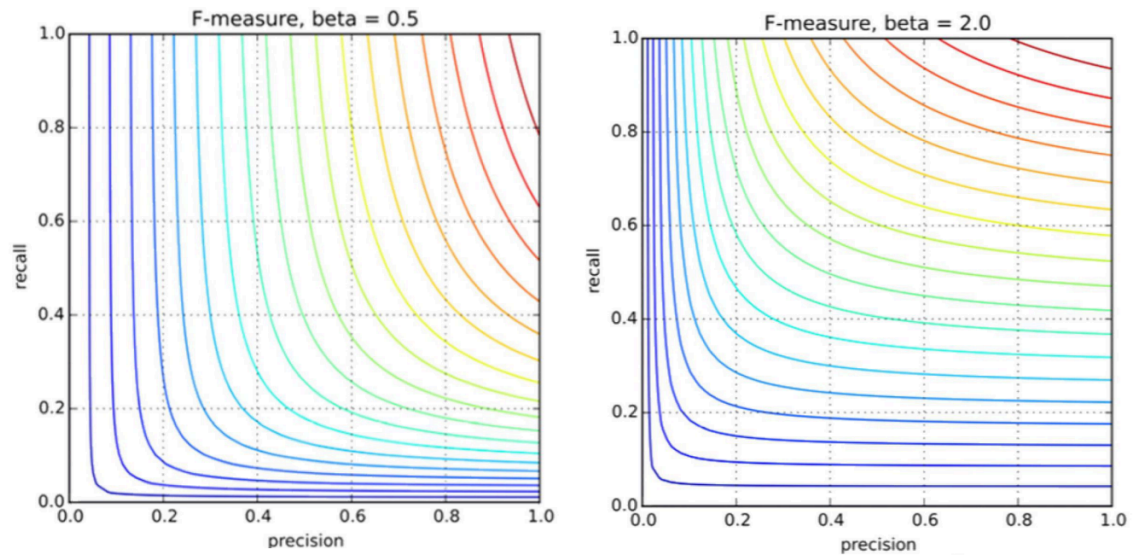
F-score показує комбінацію попередніх двох метрик. Графік нижче показує лінії одного значення F-score для різної комбінації влучності і повноти. Як бачимо, просідання будь-якого одного параметра знижує метрику F-score



Врахувати важливість влучності або повноти можна введенням додакового коефіцієнта в формулу

$$F_{\beta} = (1 + \beta^2) \frac{\text{Precision} * \text{Recall}}{\beta^2 \text{Precision} + \text{Recall}}$$

Змінюючи коефіцієнт від 0 до максимального значення будемо надавати перевагу або влучності або повноті

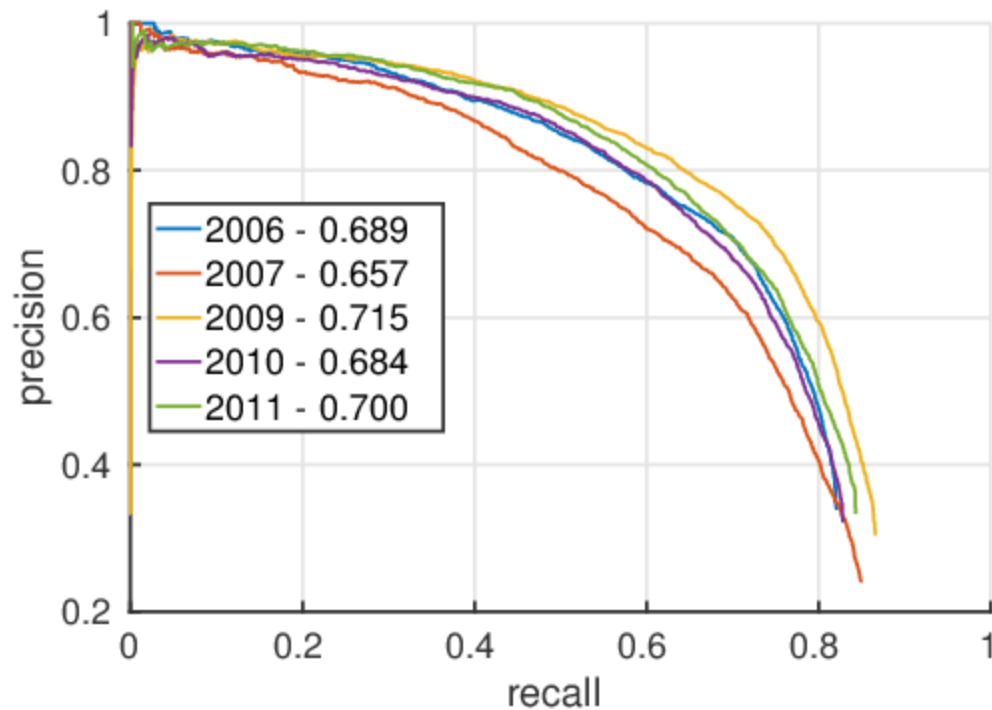


Досить популярними є також ще дві метрики **PR AUC** (площа під кривою Precision/Recall) і **ROC AUC** (площа під кривою Receiver Operating Characteristic). Для їх побудови треба розуміти, що будь-який класифікатор - це по суті регресор, предікти якого приведені до мітки класу. Наприклад, в лінійних моделях регресор для даного рядка повертає "7", залишаємо лише знак і отримуємо мітку "+1".

Першим кроком для побудови цих кривих є сортування в порядку зменшення впевненості класифікатора в позитивній мітці. Тобто в першому рядку після сортування буде найбільше значення, яке повернув регресор, а в останньому рядку буде найменше значення. Другим кроком є ітерування по отриманій таблиці і розрахунок Precision/Recall. На першій ітерації ми дивимось лише на перший рядок, на другій - на перші два, на третій - перші три і т.д. При цьому, для розрахунку Precision, TP+FP буде дорівнювати номеру ітерації, оскільки ми дивимось лише на предсказані значення **на цій ітерації**, а TP+FN - дорівнює кількості представників позитивного класу у **всьому датасеті**. Отримуємо таблицю і використовуємо колонки Precision та Recall для побудови графіка.

ID#	Parameter, concentration	Disease Yes/No	Y (sum)	N (sum)	Precision (PPV)	Recall (sensitivity)
1	33.63	Y	1	0	1.00	0.013
2	10.63	Y	2	0	1.00	0.025
3	9.90	N	2	1	0.67	0.025
4	6.87	Y	3	1	0.75	0.038
5	6.15	Y	4	1	0.80	0.050
6	6.15	Y	5	1	0.83	0.063
7	5.53	Y	6	1	0.86	0.075
8	5.08	Y	7	1	0.88	0.088
....						
....						
151	0.0041	N	77	74	0.51	0.96
152	0.0039	Y	78	74	0.51	0.98
153	0.0039	N	78	75	0.51	0.98
154	0.0039	Y	79	75	0.51	0.99
155	0.0038	N	79	76	0.51	0.99
156	0.0038	Y	80	76	0.51	1.00
157	0.0038	N	80	77	0.51	1.00
158	0.0038	N	80	78	0.51	1.00
159	0.0036	N	80	79	0.50	1.00
160	0.0036	N	80	80	0.50	1.00

Графік має вигляд:

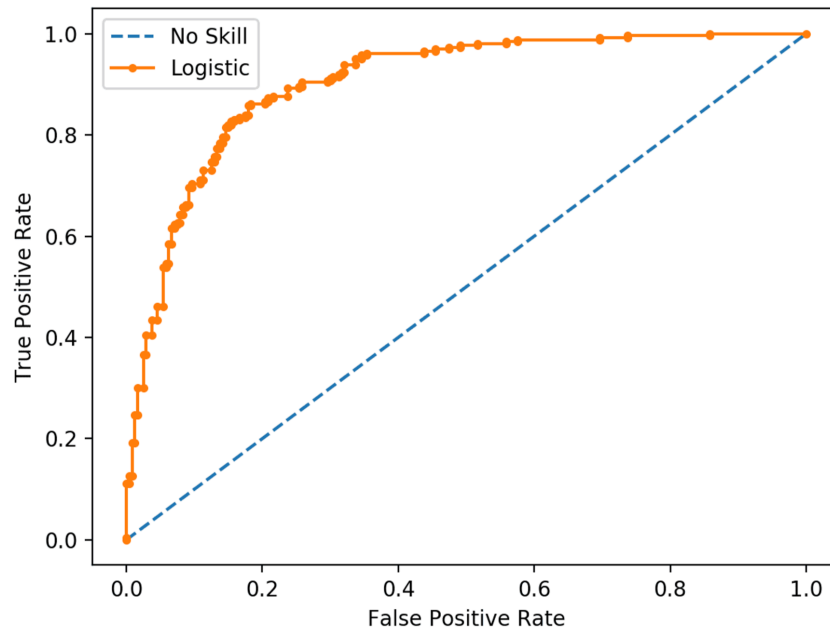


Метрика PR AUC - це площа під отриманою кривою.

Аналогічно розраховуються координати для побудови кривої ROC - True Positive Rate і False Positive Rate.

$$TPR = \frac{TP}{P + N}, FPR = \frac{FP}{N}$$

Крива ROC має вигляд:



Для обох кривих кращий випадок коли криві співпадають із верхньою стороною графіків і площа під кривою становить 1. Прийнятними є значення метрик від 0.5 до 1.0, чим більше - тим краще.

Неприйнятними є значення менше 0.5. Це значить що класифікатор помиляється більше, ніж дає правильних відповідей.

Для мультикласової класифікації для розрахунку метрик можуть бути використані два підходи: **Macro-Average** і **Micro-Average**. В першому підході ми повинні порахувати відповідні показники для кожного класу і взяти середнє значення відповідної метрики:

$$Precision_{macro} = \frac{Precision_1 + Precision_2 + \dots + Precision_N}{N}$$

При Micro-Average підході ми маємо рахувати показник одразу для всіх класів:

$$Precision_{micro} = \frac{TP_A + TP_B + TP_C}{TP_A + TP_B + TP_C + FN_A + FN_B + FN_C}$$

Macro-Average підхід дає загальне уявлення про якість класифікатора, Micro-Average підхід може підсвітити проблему, коли класифікатор віддає переважно 1-3 класи і майже не віддає представників інших класів.

Це, звичайно, не повний перелік метрик якості для класифікаторів і регресорів.

▼ Метрики якості в scikit-learn

В бібліотеці scikit-learn представлені метрики якості і для регресії, і для класифікації.

MSE

```
from sklearn.metrics import mean_squared_error
y_true = [3, -0.5, 2, 7]
y_pred = [2.5, 0.0, 2, 8]
mean_squared_error(y_true, y_pred)
y_true = [[0.5, 1], [-1, 1], [7, -6]]
y_pred = [[0, 2], [-1, 2], [8, -5]]
mean_squared_error(y_true, y_pred)
mean_squared_error(y_true, y_pred, multioutput='raw_val
ues')
mean_squared_error(y_true, y_pred, multioutput=[0.3, 0.
7])
```

MAE

```
from sklearn.metrics import mean_absolute_error
y_true = [3, -0.5, 2, 7]
y_pred = [2.5, 0.0, 2, 8]
mean_absolute_error(y_true, y_pred)
y_true = [[0.5, 1], [-1, 1], [7, -6]]
y_pred = [[0, 2], [-1, 2], [8, -5]]
mean_absolute_error(y_true, y_pred)
mean_absolute_error(y_true, y_pred, multioutput='raw_va
lues')
mean_absolute_error(y_true, y_pred, multioutput=[0.3,
0.7])
```

R2-score (цей показник також може бути як метод score в регресорі)

```

from sklearn.metrics import r2_score
y_true = [3, -0.5, 2, 7]
y_pred = [2.5, 0.0, 2, 8]
r2_score(y_true, y_pred)
y_true = [[0.5, 1], [-1, 1], [7, -6]]
y_pred = [[0, 2], [-1, 2], [8, -5]]
r2_score(y_true, y_pred,
          multioutput='variance_weighted')
y_true = [1, 2, 3]
y_pred = [1, 2, 3]
r2_score(y_true, y_pred)
y_true = [1, 2, 3]
y_pred = [2, 2, 2]
r2_score(y_true, y_pred)
y_true = [1, 2, 3]
y_pred = [3, 2, 1]
r2_score(y_true, y_pred)
y_true = [-2, -2, -2]
y_pred = [-2, -2, -2]
r2_score(y_true, y_pred)
r2_score(y_true, y_pred, force_finite=False)
y_true = [-2, -2, -2]
y_pred = [-2, -2, -2 + 1e-8]
r2_score(y_true, y_pred)
r2_score(y_true, y_pred, force_finite=False)

```

Precision, Recall, F1-score, Fbeta-score

```

import numpy as np
from sklearn.metrics import precision_score
y_true = [0, 1, 2, 0, 1, 2]
y_pred = [0, 2, 1, 0, 0, 1]
precision_score(y_true, y_pred, average='macro')
precision_score(y_true, y_pred, average='micro')
precision_score(y_true, y_pred, average='weighted')

```

```
precision_score(y_true, y_pred, average=None)
y_pred = [0, 0, 0, 0, 0, 0]
precision_score(y_true, y_pred, average=None)
precision_score(y_true, y_pred, average=None, zero_divi
sion=1)
precision_score(y_true, y_pred, average=None, zero_divi
sion=np.nan)
```

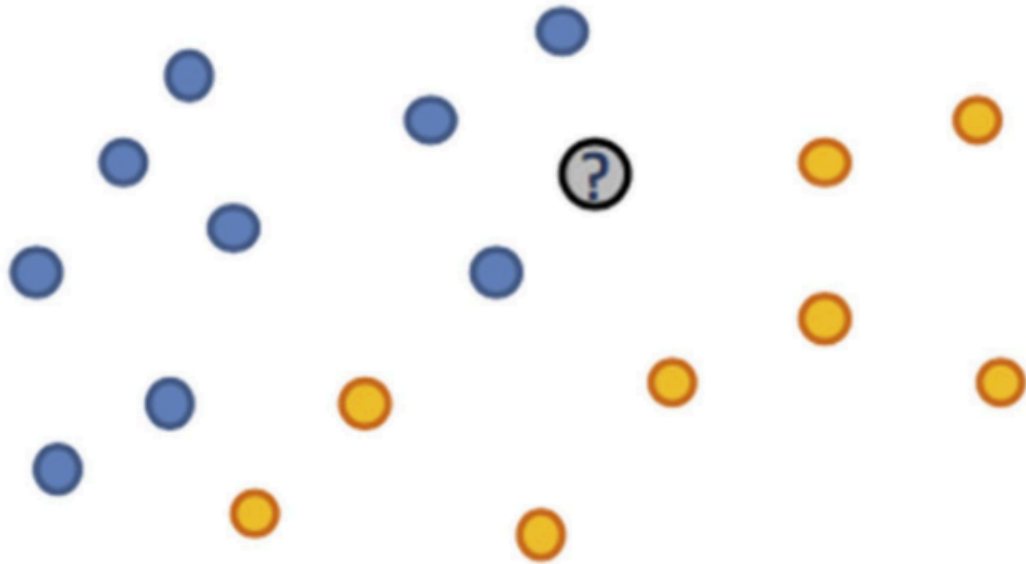
Також є методи для побудови PR та ROC кривих. І для розрахунку площ PR AUC і ROC AUC

▼ Метричні моделі

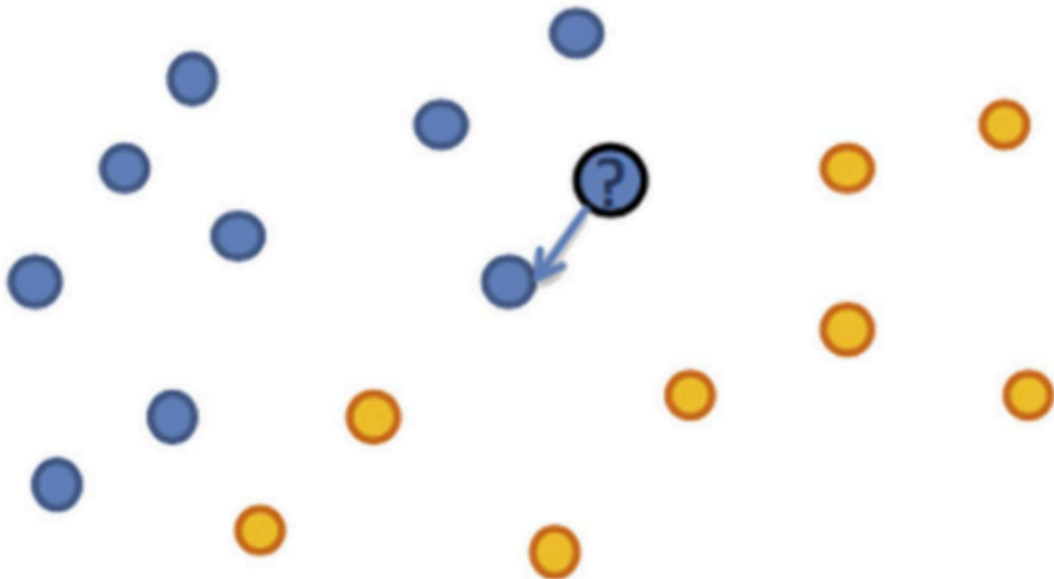
▼ Принцип передбачення в метричних моделях

Основна ідея метричних моделей базується на гіпотезі безперервності (для регресії) і гіпотезі компактності (для класифікації). Гіпотеза безперервності - якщо функція має похідну в визначеному діапазоні, то така функція буде безперервною в цьому діапазоні і будь-якій невеликій зміні аргумента відповідає невелика зміна значення функції. Формально існують функції переривчасті, ступінчасті і т.д., для яких ця гіпотеза не працює. Але, з точки зору методів прогнозування ці випадки є виключенням. Гіпотеза компактності - схожі об'єкти частіше належать одному класу, ніж різним.

Розглянемо як працює модель, що визначає клас за найближчими сусідами.

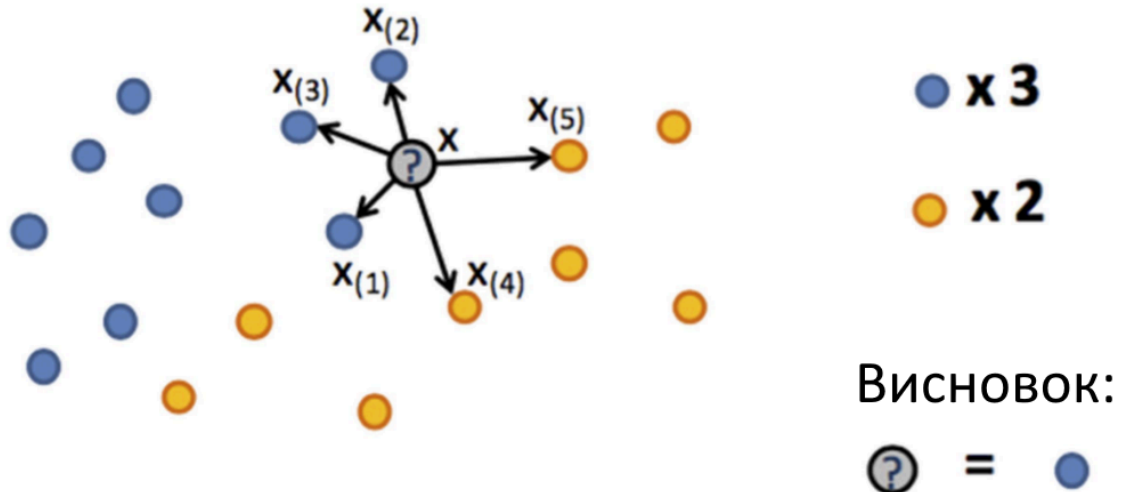


Припустимо, ми маємо набір точок двох класів: сині і жовті. До якого класу ми маємо віднести нову точку? В найпростішому випадку ми маємо знайти найближчу точку і призначити отриманий клас новій точці. В даному випадку найближчою точкою є синя.



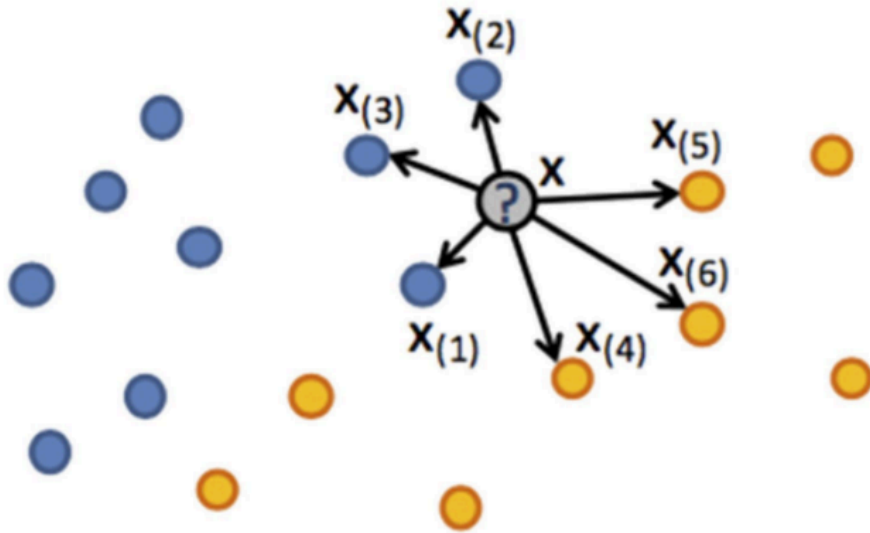
Але ми розуміємо, що найближча точка може бути викидом і насправді знаходиться посеред точок іншого класу. Тож, для підвищення надійності методу будемо орієнтуватись не на одну найближчу точку, а на кілька точок і визначати клас звичайним голосуванням. Точок якого класу більше, такий клас і призначатимемо.

Приклад класифікації ($k = 5$)



Обмеженням цього підходу є те, що для парної кількості сусідів можлива ситуація, коли точок різних класів буде однакова кількість. Плюс, ми не враховуємо близькість точок. Якщо точка одного класу ближча, ніж точка іншого класу, то вона може вносити більший вклад в визначення класу. Отже кожній точці ми можемо поставити у відповідність певний ваговий коефіцієнт.

Приклад класифікації ($k = 6$)

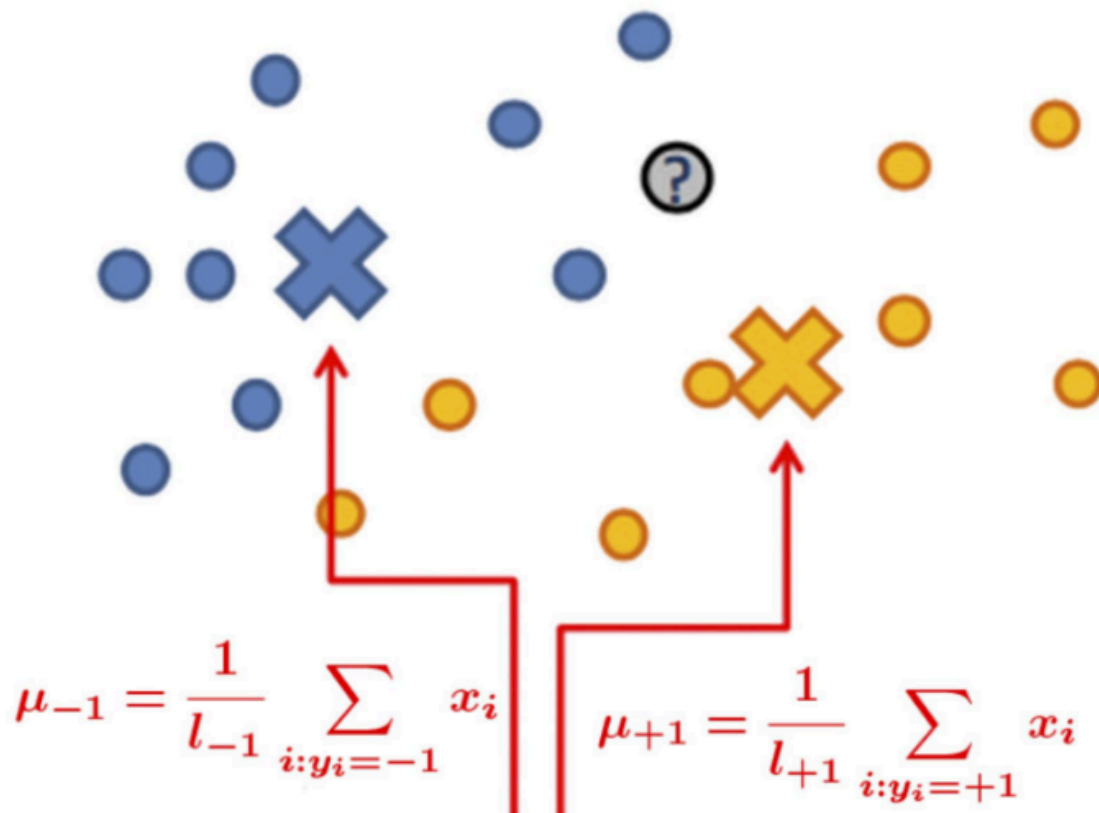


$$Z = w(x_{(1)}) + w(x_{(2)}) + w(x_{(3)})$$

$$Z = w(x_{(4)}) + w(x_{(5)}) + w(x_{(6)})$$

Точці призначимо той клас, сумарна вага якого буде більшою. Мінусом цього методу є те, що ми для кожної точки маємо обраховувати відстані до всіх точок в датасеті, щоб визначити яка з них найближча (це одна із реалізацій в scikit-learn, в інших двох дистанція розраховується для значно меншої кількості точок).

Інший підхід полягає в тому, щоб для кожного класу визначити координати центру, як середнє арифметичне відповідних координат точок кожного класу, і клас нової точки призначати базуючись на відстані до найближчого центра.

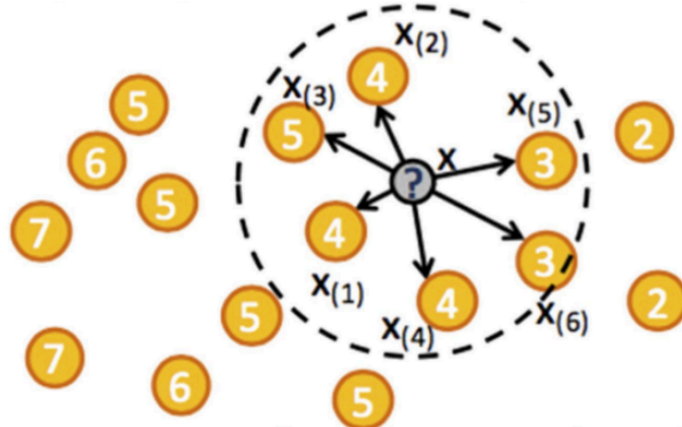


Центри класів

При такому підході новій точці буде призначений жовтий клас, оскільки жовтий центр ближче. Такий класифікатор називають центроїдним.

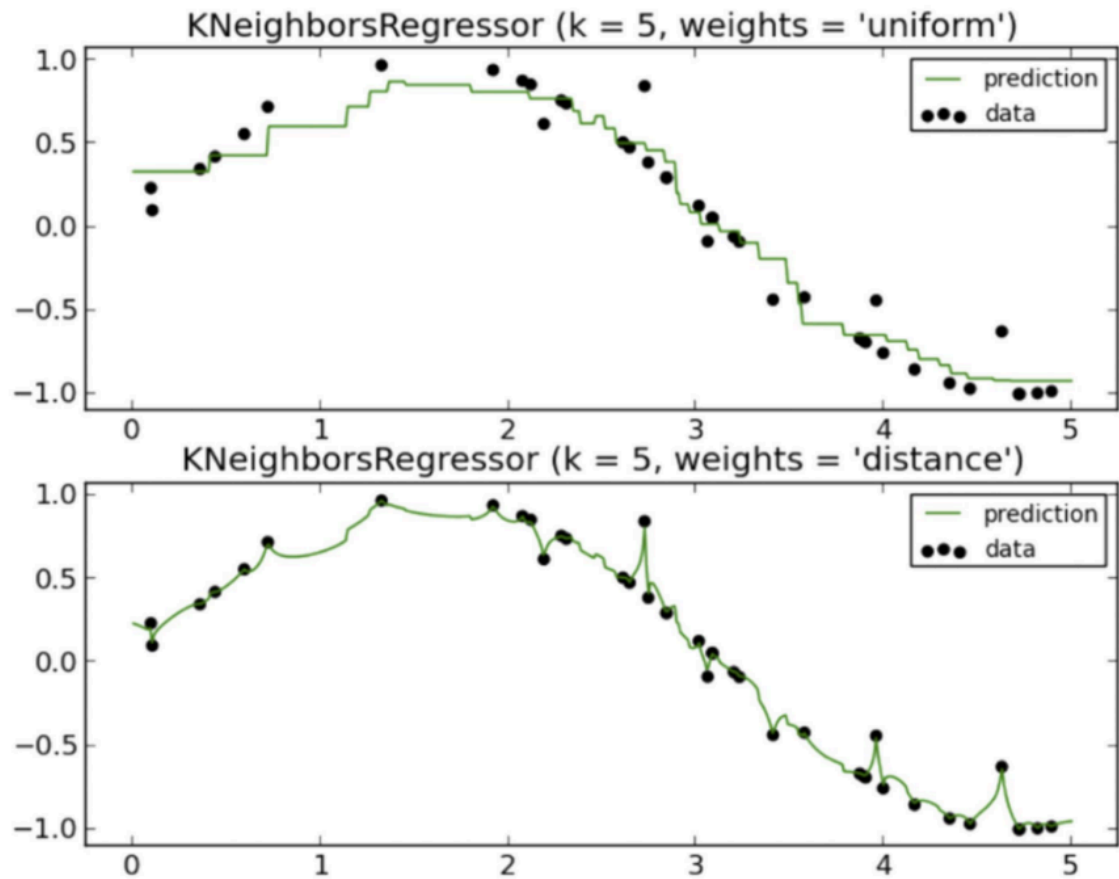
Аналогічний підхід можна застосувати і для регресії. Але, замість класу, ми будемо рахувати середнє значення найближчих точок (з врахуванням вагових коефіцієнтів або без)

Приклад регресії ($k = 6$)



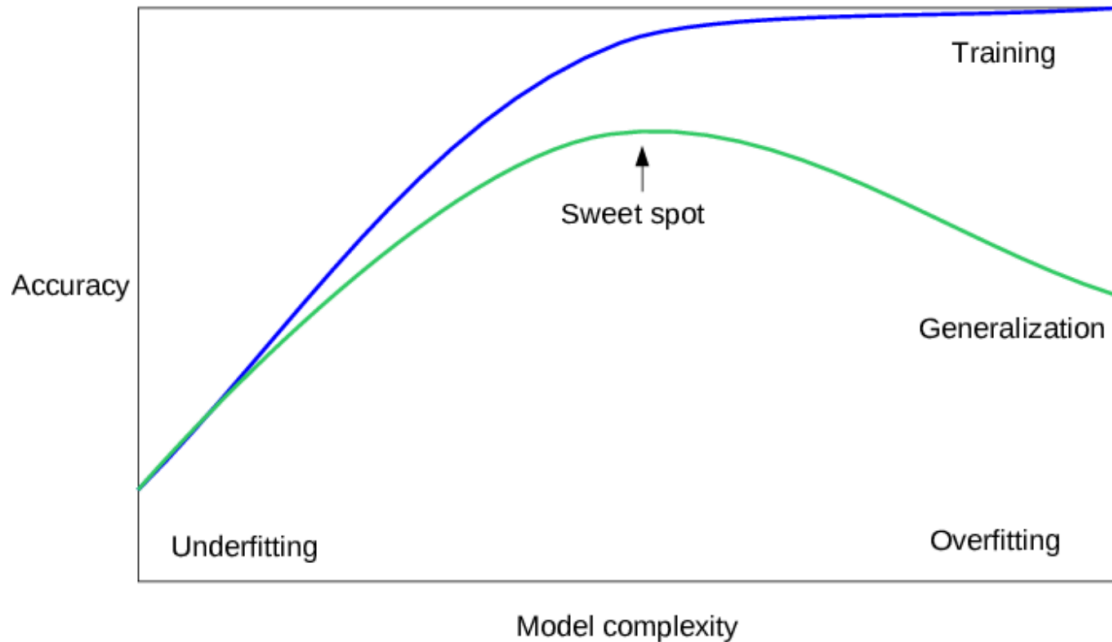
$$\textcircled{?} = \frac{4 w(x_{(1)}) + 4 w(x_{(2)}) + 5 w(x_{(3)}) + w(x_{(1)}) + w(x_{(2)}) + w(x_{(3)}) + 4 w(x_{(4)}) + 3 w(x_{(5)}) + 3 w(x_{(6)})}{w(x_{(4)}) + w(x_{(5)}) + w(x_{(6)})}$$

Однак, для регресії слід обережно відноситись до вибору методу призначення ваги. Так, якщо ваговий коефіцієнт буде обернено пропорційний до відстані, то можливе перенавчання моделі, оскільки прогнозоване значення в точці, близькій до відомої, буде наближатись до значення в цій точці. А найближча точка може бути викидом.



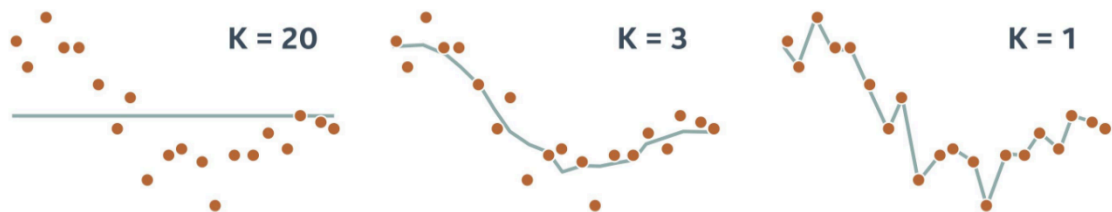
Основними гіперпараметрами методу є кількість точок і формула для розрахунку ваги точки.

Кількість точок безпосередньо впливає на перенавчання або недонавчання моделі. Згадаємо криву залежності точності моделі від її складності.



Найкращий результат умовно має бути десь на середині. Як це корелює із кількістю точок? Якщо ми оберемо велику кількість точок, то результат прогнозу буде "сильно середній", ми не відслідкуємо локальні зміни. Модель буде "простою". Якщо точка одна, то, скоріш за все, будемо мати перенавчання. Модель є "складною". На практиці рекомендують брати значення від 3 до 15 (частіше 9). Але це залежить від датасету. Наприклад, для датасету iris приблизно однакові показники метрик якості будуть в діапазоні від 25 до 70 точок.

REGRESSION WITH KNN



Залежність ваги точки від відстані може бути константною ($w=1$), залежати від відстані ($w = \frac{1}{d}$ або $w = q^d; 0 < q < 1$), залежати від індексу відстані ($w = \frac{1}{i}; w = \frac{1}{(i+a)^b}; w = q^i, 0 < q < 1$).

В загальному випадку функцію, яка не зростає на певному додатньому відрізку від 0, і використовується для розрахунку вагових коефіцієнтів називають **ядром**.

▼ Метрики відстані

Основою в метричних методах є відстань між точками. Однак ця відстань може бути розрахована по різному. В шкільній геометрії ми працюємо із евклідовим простором. Але це не єдиний варіант розрахунку відстані. Метрикою або функцією відстані будемо називати функцію, яка визначає відстань між двома точками.

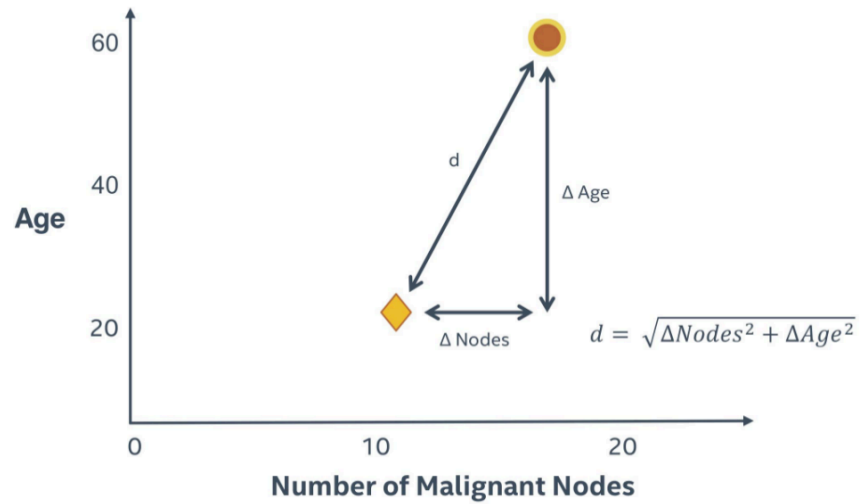
Не кожна функція може бути метрикою відстані. Сформулюємо вимоги до метрики відстані:

- відстань між точками має бути більше 0: $\rho(x, y) > 0$
- відстань між точками має бути однаковою незалежно від напрямку:
 $\rho(x, y) = \rho(y, x)$
- сума відстаней між точкою Z і двома іншими точками X, Y більше або дорівнює відстані між точками X та Y: $\rho(x, y) \leq \rho(x, z) + \rho(z, y)$

Що не може бути прикладом метрики відстані? Наприклад, відстань в часі. Між 9-ю ранку і 3-ма годинами дня - 6 годин, а між 3-ми годинами дня і 9-ю ранку - 18 годин.

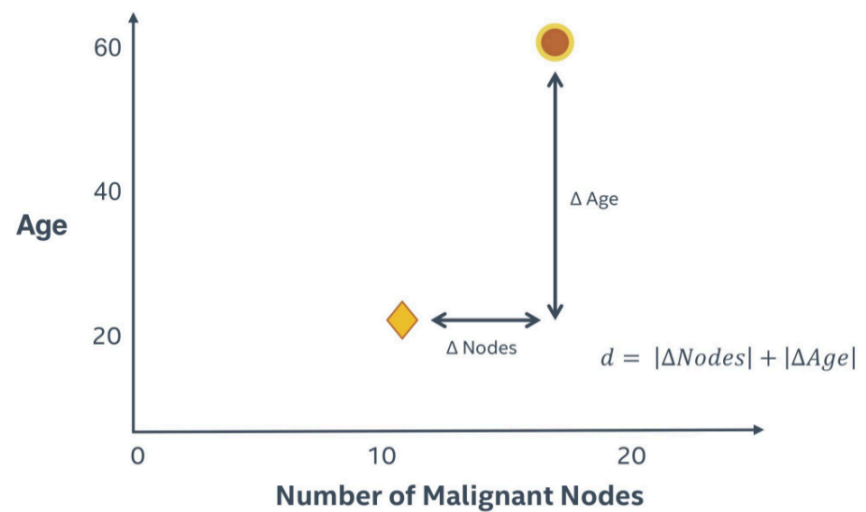
Повернемося до евклідової відстані. Це не лише про координати в просторі, а про відстань між точками в просторі ознак (фічей).

EUCLIDEAN DISTANCE (L2 DISTANCE)



Популярною є також манхеттенська відстань. Це сума абсолютних значень різниць координат.

MANHATTAN DISTANCE (L1 OR CITY BLOCK DISTANCE)



Узагальненням цих метрик є метрика Мінковського

Евклідова метрика $\sqrt{\sum_{i=1}^n (x_i - y_i)^2}$

Манхеттенська метрика $\sum_{i=1}^n |x_i - y_i|$

Метрика Мінковського $\left(\sum_{i=1}^n (|x_i - y_i|)^q \right)^{\frac{1}{q}}$

Треба пам'ятати, що при застосуванні більшості моделей, а особливо - метричних, слід масштабувати значення в колонках. Якщо значення в одній колонці змінюються від 0,001 до 0,1, а в іншій - від 100 000 до 100 000 000, то відстань за будь-якою метрикою буде порахована невірно. Найчастіше для масштабування значень фічей використовують StandardScaler.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train) # Don't cheat - fit only on training data
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test) # apply same transformation to test data

# Or better yet: use a pipeline!
from sklearn.pipeline import make_pipeline
est = make_pipeline(StandardScaler(), SGDClassifier())
est.fit(X_train)
est.predict(X_test)
```

Досить часто використовують косінусну відстань. Вона показує наскільки вектори об'єктів співнаправлені і не залежить від довжини цих векторів. Якщо вектори співнаправлені - косінусна міра дорівнює 1, якщо протилежні один одному за напрямом - -1. Ця міра найчастіше використовується для порівняння векторів слів, але може бути застосовані і в інших випадках.

$$\text{similarity} = \cos(\theta) = \frac{x \cdot y}{\|x\| \cdot \|y\|} = \frac{\sum_{i=1}^n x_i y_i}{\sqrt{\sum_{i=1}^n x_i^2} \sqrt{\sum_{i=1}^n y_i^2}}$$

▼ KNN класифікатор і регресор в scikit-learn

Бібліотека scikit-learn містить кілька варіацій класифікаторів та регресорів на основі метричних моделей. Найбільш універсальним регресором є KNeighborsRegressor

```
X = [[0], [1], [2], [3]]
y = [0, 0, 1, 1]
from sklearn.neighbors import KNeighborsRegressor
neigh = KNeighborsRegressor(n_neighbors=2)
neigh.fit(X, y)
print(neigh.predict([[1.5]]))
```

Серед класифікаторів виділимо KNeighborsClassifier та NearestCentroid. Перший реалізує класичний підхід на основі найближчих точок (з врахуванням вагів чи без, різні метрики відстаней, підходи до відбору точок), другий є реалізацією центроїдного класифікатора

```
X = [[0], [1], [2], [3]]
y = [0, 0, 1, 1]
from sklearn.neighbors import KNeighborsClassifier
neigh = KNeighborsClassifier(n_neighbors=3)
```

```
neigh.fit(X, y)
print(neigh.predict([[1.1]]))
print(neigh.predict_proba([[0.9]]))

from sklearn.neighbors import NearestCentroid
import numpy as np
X = np.array([[-1, -1], [-2, -1], [-3, -2], [1, 1], [2, 1], [3, 2]])
y = np.array([1, 1, 1, 2, 2, 2])
clf = NearestCentroid()
clf.fit(X, y)
print(clf.predict([[-0.8, -1]]))
```